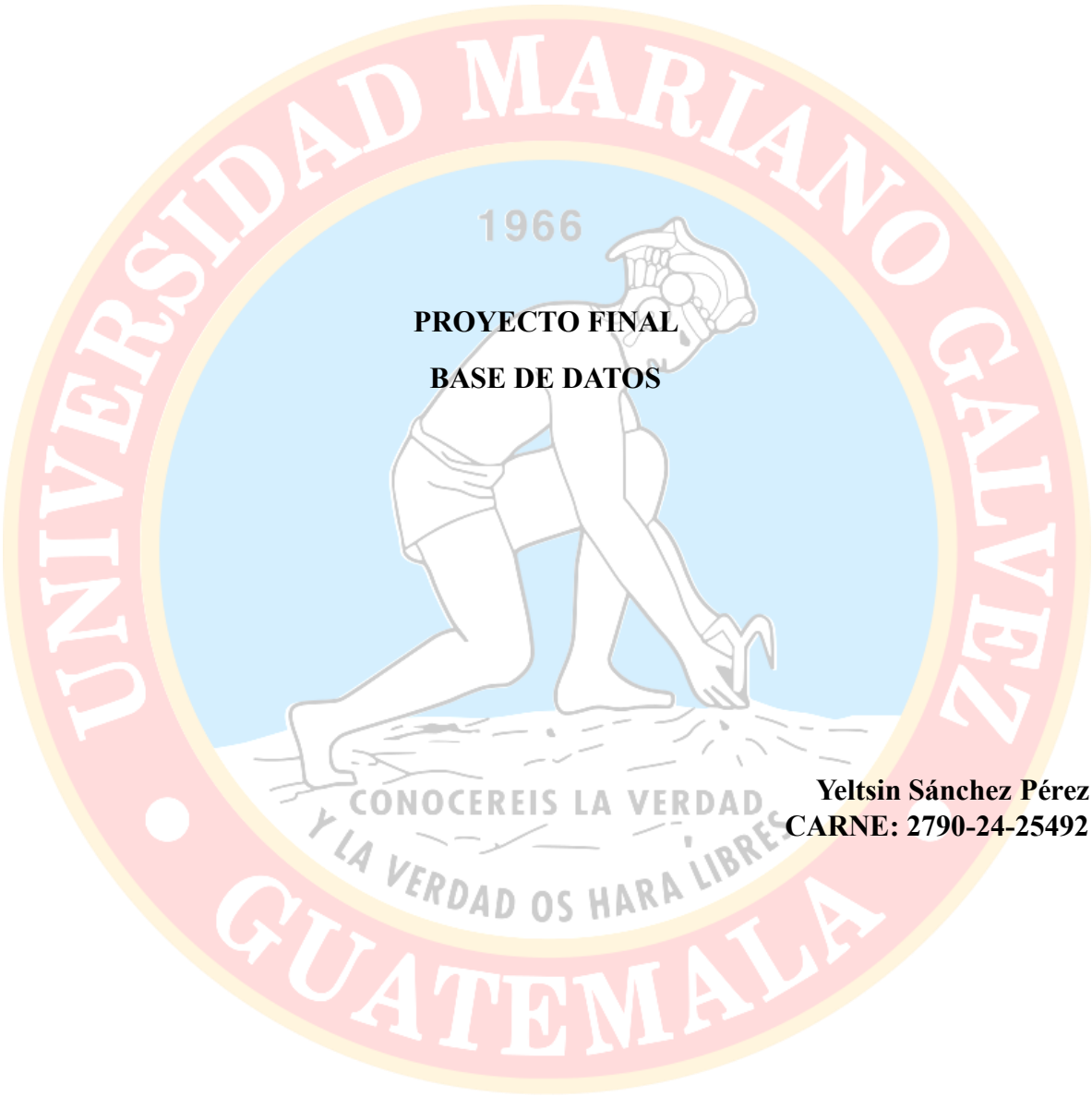
UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA
FACULTAD DE INGENIERIA



PROYECTO FINAL

BASE DE DATOS

Yeltsin Sánchez Pérez
CARNE: 2790-24-25492

1. Descripción del problema

La tienda de tecnología objeto del proyecto lleva sus ventas y el control de existencias en libretas y hojas de cálculo. Esa forma de trabajar ha producido problemas concretos: el stock registrado casi nunca coincide con el físico porque no hay un registro confiable de las entradas de mercancía; los precios se calculan a mano y aparecen duplicados en distintas hojas, lo que origina errores de ganancia; el cierre de caja se hace al final del día sin un respalde de los movimientos; y no existe forma de responder a tiempo preguntas como cuánto se vendió el mes pasado o cuál fue el producto más vendido.

El proyecto resuelve esa situación con un sistema web que centraliza productos, compras, ventas, caja y reportes en una base de datos relacional. Toda operación que modifica información se ejecuta con validaciones y transacciones, de modo que una venta incompleta o sin existencias no deje la base en un estado inconsistente.

2. Objetivo

Diseñar e implementar una solución de software que administre la información de la tienda mediante una base de datos relacional normalizada de veinte tablas y una API que permita consultar, crear, actualizar y eliminar registros. La solución debe ejecutar sus operaciones de negocio con procedimientos almacenados y transacciones, y ofrecer reportes que utilizan funciones de agrupación de SQL.

3. Alcance

El sistema cubre el punto de venta con carrito de compras, validación de stock al momento de cobrar y registro del cobro por forma de pago; el inventario con fichas de productos, entradas de mercancía, kardex y promociones; la caja con turnos, movimientos de ingreso y egreso, y cierre con saldo calculado; las devoluciones, que repone el inventario; y los reportes de ventas por categoría, por mes y de productos más vendidos. La misma información se expone por una API JSON, usada también por el teléfono que funciona como escáner inalámbrico.

Quedan fuera del alcance la facturación electrónica, la gestión de varios almacenes y las ventas a crédito.

4. Usuarios y necesidades funcionales

Usuario	Rol	Acceso
Administrador	`admin`	Todo el sistema: catálogos, productos, entradas, caja, devoluciones, reportes, usuarios y API.
Vendedor / Cajero	`empleado`	Ventas, buscador, escáner y consulta de reportes.

Los accesos de prueba son `admin / admin123` y `vendedor / vendedor123`; las contraseñas se guardan con hash y no se almacenan en texto plano. Las necesidades funcionales que se derivan de los usuarios son: registrar y actualizar productos con código, código de barras, costo, margen, precio y stock; ingresar mercancía que actualice el existente; vender por escáner o teclado con el stock validado en el momento; cobrar en efectivo, tarjeta o transferencia y cerrar el turno de caja; registrar devoluciones que repone el inventario; consultar reportes de ventas; y exponer la información por API.

5. Reglas de negocio

1. Cada producto tiene un código único: no se puede dar de alta dos veces el mismo código, y el nombre y el precio son obligatorios.
2. No se vende más de lo que hay en stock. Si algún ítem de la venta supera el existente, la operación completa se revierte con ROLLBACK y no queda registrada.
3. Toda venta modifica el inventario dentro de la misma transacción: cabecera, detalle, descuento de stock, kardex, pago y movimiento de caja se confirman juntos o fallan juntos.
4. Una entrada de mercancía siempre aumenta el stock y queda registrada en el kardex con su fecha y la referencia de la compra.
5. Una devolución repone el stock únicamente de los productos y en las cantidades que pertenecen a esa venta; si el producto no estaba en la venta, se rechaza.
6. El cierre de caja es una sola operación: saldo final igual a saldo inicial más ingresos menos egresos del turno. Cerrado el turno, no admite más movimientos.
7. Solo permanece abierto un turno de caja a la vez, para poder asociarle los cobros del día.

6. Modelo de datos: las 20 tablas

La base de datos se diseñó con veinte tablas relacionadas mediante claves primarias y foráneas. Ninguna se creó para alcanzar la cantidad mínima: todas representan una

entidad o un evento real del negocio y tienen una función definida dentro del flujo de trabajo.

Módulo	Tablas	Función
Catálogos	`roles`, `categorias`, `marcas`, `proveedores`, `clientes`, `formas_pago`, `cajas`	Datos maestros que se referencian por FK desde el resto del modelo: permisos, agrupaciones de productos, quién nos vende, a quién vendemos, cómo se paga y la caja física.
Usuarios	`usuarios`	Quién opera el sistema, con contraseña encriptada y rol tomado de `roles`.
Inventario	`productos`, `entradas`, `movimientos_inventario`, `promociones`	Ficha de artículos, compras a proveedores, kardex de trazabilidad de cada movimiento y descuentos con vigencia.
Ventas	`ventas`, `detalle_ventas`, `pagos_venta`, `devoluciones`, `detalle_devoluciones`, `ventas_pendientes`	Cabecera y líneas de cada venta, forma y monto del pago, devoluciones con su detalle y el carrito del escáner pendiente de cobro.
Caja	`turnos_caja`, `movimientos_caja`	Apertura y cierre de cada turno con su saldo, y los ingresos, egresos y ventas que lo modifican.

Todas las tablas se ven en el sistema: catorce tienen pantalla propia (catálogos, inventario, entrada, kardex, promociones, caja, ventas, devoluciones, usuarios y reportes), y las restantes se muestran dentro de la pantalla donde corresponde: `detalle_ventas` y `pagos_venta` en el detalle de cada venta, `detalle_devoluciones` en el historial de devoluciones, `movimientos_caja` en la tarjeta del turno, `cajas` al abrir el turno y `ventas_pendientes` cuando el escáner deja un carrito en Caja. Además, nueve tablas exponen su CRUD por la API y las veinte aparecen en el archivo `schema.sql` con sus restricciones.

7. Normalización

El modelo se normalizó hasta la tercera forma normal. En primera forma normal no existen columnas multivalor ni repetidas: los ítems de una venta se guardan en `detalle_ventas` y no en columnas como “producto1, producto2”, y los pagos viven en `pagos_venta` con un valor atómico por fila. En segunda forma normal no hay dependencias

parciales: las tablas de detalle tienen clave simple y todos sus atributos dependen de esa clave. En tercera forma normal no quedan dependencias transitivas: el producto no almacena el nombre de la categoría ni de la marca, solo sus identificadores con una foránea a `categorias` y `marcas`; el turno de caja no copia el nombre del usuario, lo referencia desde `usuarios`.

Para el documento se analizaron en detalle tres tablas representativas con sus dependencias funcionales. En `detalle_ventas`, la pareja (venta, producto) determina cantidad, precio unitario, total y ganancia; el total se almacena para conservar el importe histórico aunque después cambie el precio de lista, y la ganancia se calcula al momento de la venta sin guardar el costo del producto, lo que evita una transitiva. En `productos`, el código determina el resto de los atributos; el precio deriva de costo y margen y se recalcula solo cuando esos campos se editan. En `turnos_caja`, el saldo final no se suma en cada consulta: se calcula una única vez al cerrar el turno mediante el procedimiento almacenado y se guarda como cierre de caja, conservando la auditoría. El análisis completo está en el archivo `FASE_2.md` del repositorio.

8. API

La API está escrita en Python con Flask y se conecta directamente a Microsoft SQL Server con SQLAlchemy y el driver `pyodbc`. Implementa operaciones CRUD en nueve tablas —categorías, marcas, proveedores, clientes, productos, formas de pago, entradas, ventas y devoluciones—, lo que supera el mínimo de cinco que pide la rúbrica. Cada operación tiene su endpoint, valida los datos recibidos y devuelve el código HTTP que corresponde: 200 al consultar, 201 al crear, 400 cuando la información es inválida, 404 si el registro no existe y 409 ante conflictos de clave duplicada. La lista completa de endpoints y parámetros se documenta en `FASE_2.md`.

Las credenciales del sistema no están escritas en el código fuente: la conexión a SQL Server usa autenticación de Windows y el archivo `.env` se ignora en Git, por lo que nunca se sube al repositorio.

9. Procedimientos almacenados y transacciones

Procedimiento	Qué hace	Transacción
`SP_REGISTRARVENTA`	Registra la venta y actualiza existencias: crea cabecera, detalle y pago, descuenta el stock,	Escrito con `BEGIN TRY` / `BEGIN CATCH`: valida el stock, y si algún ítem no alcanza lanza `THROW`, hace ROLLBACK en el CATCH y no se

	escribe el kardex y el movimiento de caja.	guarda nada; si todo está correcto, COMMIT.
`SP_CERRARCAJA`	Calcula el saldo final del turno sumando ingresos y restando egresos, y cambia el estado a cerrado.	Opera sobre `MOVIMIENTOS_CAJA` y `TURNOS_CAJA` en una sola transacción; un turno cerrado queda inmutable.
`SP_APLICARDEVOLUCION`	Valida que las unidades devueltas no superen lo vendido, inserta la devolución y su detalle, repone el stock y registra el kardex.	Mismo patrón TRY/CATCH: validaciones con `THROW` y COMMIT o ROLLBACK completo si algún dato no cumple.

Los tres procedimientos se invocan desde la API —`POST /api/ventas` y `POST /api/devoluciones`— y desde las pantallas de Caja y Devoluciones, de modo que la interfaz y la API comparten la misma lógica de negocio. El detalle del envío de los renglones de la venta se hace con un tipo de tabla (`TIPODETALLEVENTA`) y el identificador generado se devuelve por un parámetro `OUTPUT`.

10. Reportes con funciones de agrupación

El sistema ofrece tres reportes construidos con agregaciones de SQL, accesibles desde la pantalla de reportes y desde endpoints de la API:

- **Ventas por categoría**: suma de montos y cantidad de ventas agrupada con `GROUP BY` sobre el nombre de la categoría.
- **Ventas por mes**: total y número de ventas agrupado por mes con `CONVERT` y `SUM` / `COUNT`.
- **Top 10 de productos más vendidos**: suma de unidades vendidas por producto con `GROUP BY` y `TOP 10`.

Los tres responden a necesidades reales de la tienda (saber qué se vende, en qué periodo y qué producto conviene reponer) y se ejecutan en el motor de base de datos, no en el código de Python.

11. Datos de prueba

El repositorio incluye el script `schema.sql`, que crea las veinte tablas con sus restricciones y datos iniciales de catálogo, y `procedimientos.sql`, con los tres procedimientos almacenados. Además, el archivo `sembrar_demo.py` siembra una escena completa de prueba: dos proveedores, dos clientes, seis productos con sus entradas y kardex, tres

ventas —dos del mes en curso y una del mes anterior para que los reportes muestren dos periodos—, una devolución, un turno de caja cerrado y una promoción activa. Con eso, las pantallas de kardex, reportes y caja se visualizan con contenido real. El script `limpiar_demo.sql` permite volver la base a su estado inicial.

12. Instalación y ejecución

1. Instalar dependencias: `python -m pip install -r requirements.txt`
2. Crear la base de datos: `sqlcmd -S (local)\SQLEXPRESS -E -i schema.sql` y `sqlcmd -S (local)\SQLEXPRESS -E -i procedimientos.sql`
3. La conexión usa autenticación de Windows; si la instancia tiene otro nombre se ajusta en el archivo `.env`.
4. Sembrar datos de demostración: `python sembrar_demo.py`
5. Arrancar: `python app.py` y abrir `https://localhost:5000`

Requisitos: Python 3.12, Microsoft SQL Server 2022 Express con el driver ODBC 18, y el navegador Google Chrome o Microsoft Edge.

13. Conclusiones

El proyecto cumplió los objetivos planteados: la información de la tienda ya no depende de hojas de cálculo dispersas, sino de una base relacional con veinte tablas en tercera forma normal, integridad referencial y restricciones que impiden datos inválidos. Las operaciones de negocio críticas —venta, cierre de caja y devolución— se ejecutan como procedimientos almacenados con transacciones, de modo que una operación fallida no deja cambios a medias. La API permite consultar y administrar la información desde cualquier dispositivo y los reportes responden en segundos preguntas que antes tomaban horas de revisar manualmente.

Como trabajo futuro se podrían incorporar facturación electrónica, control de varios almacenes y un módulo de compras a crédito, todos ellos compatibles con el modelo actual sin necesidad de rediseñarlo.

14. Repositorio

El código fuente completo, el script SQL, la documentación del diseño y las instrucciones de ejecución se encuentran en:

https://github.com/userysanch/the_tech_house

Archivos de apoyo: `FASE_1.md` *(estructura y configuración),* `FASE_2.md` *(modelo entidad-relación, normalización, API y procedimientos),* `FASE_3.md` *(mapa de las 20 tablas a pantallas) y* `GUION_DEMO.md` *(guía de demostración).*